

개발 일지

1) 작업 내용 (What I Did)

구분	내용
작업 내용 (What I Did)	프로젝트 범위 정의 및 주소/핀맵 확정: AXI4-Lite 버스 기반 SoC 설계와 custom AXI IP의 설계·검증을 메인 주제로 확정하고, 대표 응용 기능으로 SPI RFID 근태관리를 선정하였다. 여기에 SR04 초음파 거리 측정, I2C LCD 출력, Stopwatch 기능을 더해 SW[2:0] 기반 모드로 구성하기로 하였으며, RFID(SPI)가 JA1~JA4를 선점하므로 SR04는 JB3/JB4로 배정하여 핀 충돌을 사전에 예방하였다.

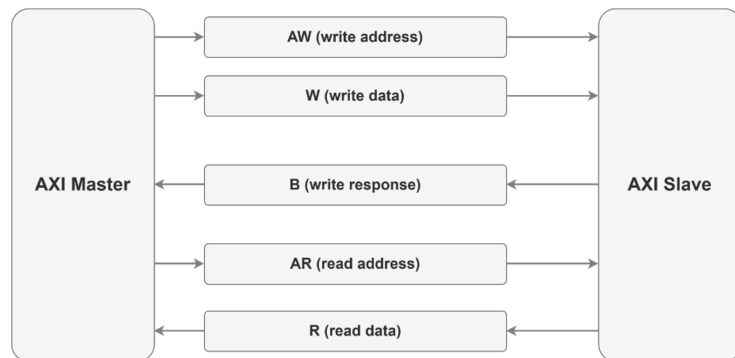


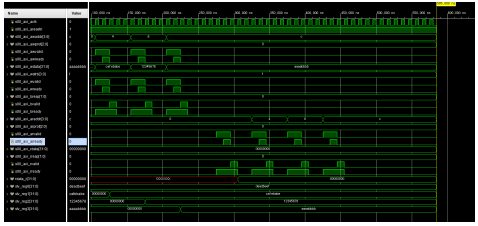
그림1. AXI4 Bus Protocol

프로토콜 채널 및 기존 템플릿 분석: AXI4-Lite의 5개 채널 (AW/W/B/AR/R)과 valid/ready 핸드셰이크 규칙을 파형 수준으로 정밀 정리하였다. 특히 master가 valid를, slave가 ready를 구동하며 1 클럭 내에 전송이 완료되는 규칙과, write 완료 후 bresp가 돌아오는 흐름을 도식화하였다. 이와 함께 Xilinx AXI4-Lite slave template 코드를 분석하여 awready/wready 생성, wstrb 기반의 byte lane별 레지스터 갱신, araddr 디코딩 흐름을 구조적으로 파악하였다.

2) 회고 및 개선 사항

구분	내용
Keep (잘한 점, 유지할 점)	구현 전에 프로토콜 규칙(AXI handshake, RC522 frame)을 정해 두어, 이후 설계·검증에서 참조 기준이 흔들리지 않았다.
Try (새롭게 시도할 점, 개선할 점)	AXI4-Lite 와 AXI4 full 의 차이(burst, outstanding, ID)를 표로 정리해 두었으면 좋았을 거 같다.

3) 이슈 및 해결 과정

구분	내용
Trouble (이슈/장애 및 해결 과정)	 AXI4 템플릿 Read Data(rdata) 출력 누락 : AXI4 템플릿 테스트중 rdata가 나오지 않는 상황 발생 → 코드상에 레지스터에서 rdata를 받는 부분이 지워져있어 그부분 다시 추가하여 해결 RC522 SPI 통신 Address Byte 포매팅 규칙 정립 : 통신 규격을 분석하는 과정에서 RC522의 레지스터 읽기/쓰기 명령에 사용되는 주소 바이트 구성 규칙을 이해하는 데 어려움이 있어 초기 설계 방향에 다소 혼선이 있었다. 단순히 레지스터 주소를 전송하는 것이 아니라, Write 명령에서는 $(reg \ll 1) \& 0x7E$ 형태로 주소를 구성하고, Read 명령에서는 여기에 최상위 비트(Read 비트)를 설정한 $((reg \ll 1) \& 0x7E) 0x80$

구분	내용
	형식으로 전송해야 한다는 규칙을 데이터시트를 재분석하는 과정에서 명확히 이해하였다.